

Git and Version Control System – Complete Hands-On Lab (HOL)

Introduction

In modern software development, multiple developers work on the same application, modify files continuously, fix bugs, add features, and deploy applications frequently. Without a proper tracking mechanism, managing changes becomes extremely difficult. Developers may overwrite each other's work, lose important code, or fail to identify who made a particular change.

To solve these problems, Version Control Systems (VCS) were introduced.

A Version Control System helps developers track changes in files, maintain history, collaborate with teams, restore older versions, and manage software development efficiently.

Git is the most popular distributed version control system used across the software industry today.

This document explains:

- What is Version Control System
 - Types of Version Control Systems
 - Complete Git concepts in detail
 - Git architecture and workflow
 - Installation process
 - Hands-On Lab using real Git commands
 - Detailed explanation of each command
-

What is a Version Control System (VCS)?

A Version Control System is a software tool that tracks changes made to files over time. It stores every modification in the form of versions so developers can easily:

- Track changes
- Restore previous versions
- Compare modifications
- Collaborate with teams
- Manage multiple versions of software
- Avoid code conflicts
- Maintain backup of source code

Suppose a developer modifies an application today and another developer modifies the same application tomorrow. Without version control, the old code may be lost forever.

With VCS:

- Every change is recorded
- Previous versions are preserved
- Developers can identify who changed what
- Teams can work simultaneously
- Code can be restored anytime

Version control systems are heavily used in:

- Software development
- Data engineering
- DevOps
- Machine learning projects
- Cloud infrastructure automation
- Documentation management

Why Version Control System is Important

A Version Control System provides multiple advantages.

Change Tracking

Every modification made in a file is recorded.

Collaboration

Multiple developers can work together without overwriting each other's code.

Backup and Recovery

Older versions can be restored anytime.

Audit and History

Developers can identify who changed the code and why.

Branching and Experimentation

New features can be developed safely without affecting production code.

Parallel Development

Teams can work independently on different modules.

Types of Version Control Systems

Version Control Systems are mainly divided into three categories.

1. Local Version Control System

In Local VCS, all version history is stored on the local machine.

The developer manually maintains copies of files.

Example:

- File_v1
- File_v2
- File_final
- File_final_latest

Problems:

- Difficult to manage
- High chance of data loss
- No collaboration support
- No centralized history

Use Case:

- Personal projects
 - Small standalone tasks
-

2. Centralized Version Control System (CVCS)

In Centralized VCS, a central server stores all versions.

Developers connect to the central repository to:

- Pull code
- Commit changes
- Track versions

Examples:

- SVN (Subversion)
- Perforce
- CVS

Architecture:

- One central server
- Multiple clients

Advantages:

- Easy administration
- Centralized control
- Simple collaboration

Disadvantages:

- Server failure affects everyone
- Requires network connection
- Single point of failure

Use Cases:

- Enterprise applications
- Controlled development environments

3. Distributed Version Control System (DVCS)

In Distributed VCS, every developer has a complete copy of the repository.

Each machine contains:

- Full codebase
- Complete commit history
- Branches
- Tags

Examples:

- Git
- Mercurial

Advantages:

- No single point of failure
- Faster operations
- Offline work support
- Better collaboration
- Full backup available locally

Disadvantages:

- Slightly complex for beginners

Use Cases:

- Modern software development
- DevOps pipelines
- Open-source projects
- Cloud-native applications
- Enterprise engineering teams

Git is a Distributed Version Control System.

What is Git?

Git is an open-source distributed version control system developed by Linus Torvalds.

Git is designed to:

- Track source code changes
- Support distributed development
- Enable collaboration
- Maintain version history
- Provide fast branching and merging

Git is extremely fast because most operations are performed locally.

Git maintains snapshots instead of storing files repeatedly.

Every commit in Git represents a complete snapshot of the project at that time.

Important Features of Git

Distributed Architecture

Every developer has a full copy of the repository.

Fast Performance

Most operations are local.

Branching Support

Git supports lightweight branches.

Data Integrity

Git uses SHA hashing to protect data.

Offline Work

Developers can commit changes without internet.

Collaboration

Teams can work efficiently together.

Git Architecture

Git mainly works with three areas.

1. Working Directory

This is the actual project directory where developers create and modify files.

Example:

app.py config.txt README.md

2. Staging Area (Index)

The staging area acts as an intermediate layer.

Developers select which changes should go into the next commit.

The command used:

```
git add
```

3. Local Repository

The local repository stores committed versions permanently.

The command used:

```
git commit
```

Git Workflow

The normal Git workflow is:

Working Directory → Staging Area → Local Repository

Steps:

1. Create or modify files
 2. Check status
 3. Add files to staging area
 4. Commit changes
 5. Push to remote repository
-

Difference Between Git and GitHub

Git and GitHub are different.

Git

Git is a version control tool installed locally.

GitHub

GitHub is a cloud platform that hosts Git repositories.

Examples of Git hosting platforms:

- GitHub
 - GitLab
 - Bitbucket
 - Azure DevOps
-

Git Installation Process

Install Git on Linux (Ubuntu)

Update packages:

```
sudo apt update
```

Install Git:

```
sudo apt install git -y
```

Verify installation:

```
git --version
```

Install Git on CentOS/RHEL

```
sudo yum install git -y
```

Verify:

```
git --version
```

Install Git on Windows

Step 1:

Download Git from the official website:

<https://git-scm.com/>

Step 2:

Run the installer.

Step 3:

Choose default options.

Step 4:

Complete installation.

Step 5:

Open Git Bash.

Verify:

```
git --version
```

Install Git on macOS

Using Homebrew:

```
brew install git
```

Verify:

```
git --version
```

Configure Git

After installation, configure username and email.

```
git config --global user.name "Your Name"
```

```
git config --global user.email "your_email@example.com"
```

Check configuration:

```
git config --list
```

Create First Git Repository

Create directory:

```
mkdir git-demo
```

Move into directory:

```
cd git-demo
```

Initialize repository:

```
git init
```

This creates a hidden .git folder.

The .git folder stores:

- Commit history
 - Branches
 - Configuration
 - Metadata
-

Complete Hands-On Lab (HOL)

Step 1 – Create app.py File

Command:

```
vi app.py
```

Explanation:

This command opens the VI editor and creates the file app.py.

Inside the file, write:

```
print("Hello Git")
```

Save and exit:

- Press ESC
- Type `:wq`
- Press Enter

Step 2 – Check Git Status

Command:

```
git status
```

Explanation:

This command checks:

- Modified files
- Untracked files
- Staged files
- Current branch

Output Meaning:

Git will show app.py as an untracked file.

Untracked means Git is aware of the file but not yet managing it.

Step 3 – Add File to Staging Area

Command:

```
git add app.py
```

Explanation:

This command moves app.py from the working directory to the staging area.

Now Git is preparing this file for commit.

Step 4 – Verify Status Again

Command:

```
git status
```

Explanation:

Git now shows:

- app.py is staged
 - Ready for commit
-

Step 5 – Remove File from Staging Area

Command:

```
git restore --staged app.py
```

Explanation:

This command removes app.py from the staging area.

Important:

- File changes are NOT deleted
 - Only staging is removed
 - File remains in working directory
-

Step 6 – Check Status Again

Command:

```
git status
```

Explanation:

Git now shows app.py as untracked again.

Step 7 – Commit File Directly

Command:

```
git commit -m "first commit" app.py
```

Explanation:

This command creates a commit.

Commit means:

- Permanent snapshot of changes
- Stored in local repository
- Recoverable anytime

The message:

```
"first commit"
```

helps identify the purpose of the commit.

Important:

If the file is not staged properly, Git may throw warnings.

Best practice:

Always use:

```
git add
```

before commit.

Step 8 – Check Repository Status

Command:

```
git status
```

Explanation:

This verifies whether the working tree is clean.

If everything is committed:

```
nothing to commit, working tree clean
```

Step 9 – List Files

Command:

```
ls
```

Explanation:

Displays files available in the current directory.

Step 10 – Modify app.py

Command:

```
vi app.py
```

Add another line:

```
print("Second Update")
```

Save and exit.

Step 11 – Add File Again

Command:

```
git add app.py
```

Explanation:

Git stages the modified version.

Step 12 – Remove from Staging Again

Command:

```
git restore --staged app.py
```

Explanation:

This unstages the file again.

Changes remain safe.

Step 13 – Check Status

Command:

```
git status
```

Explanation:

Git shows modified file.

Step 14 – Add File Again

Command:

```
git add app.py
```

Explanation:

Stages the modified file again.

Step 15 – Verify Staging

Command:

```
git status
```

Explanation:

Git confirms file is staged.

Step 16 – Remove from Staging Again

Command:

```
git restore --staged app.py
```

Explanation:

Unstages changes.

Step 17 – Check Status Again

Command:

```
git status
```

Explanation:

Git displays modified but unstaged file.

Step 18 – Add All Files

Command: